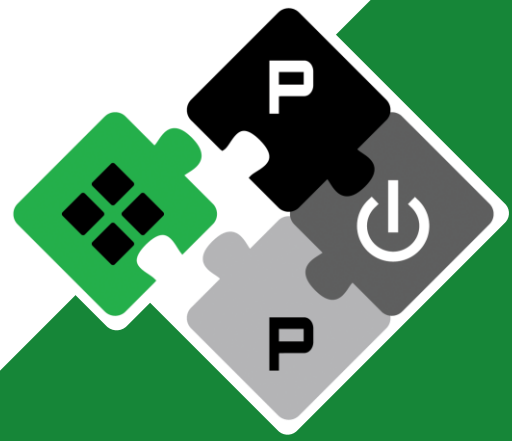
ETH zürich ALMA MATER STUDIORUM UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch



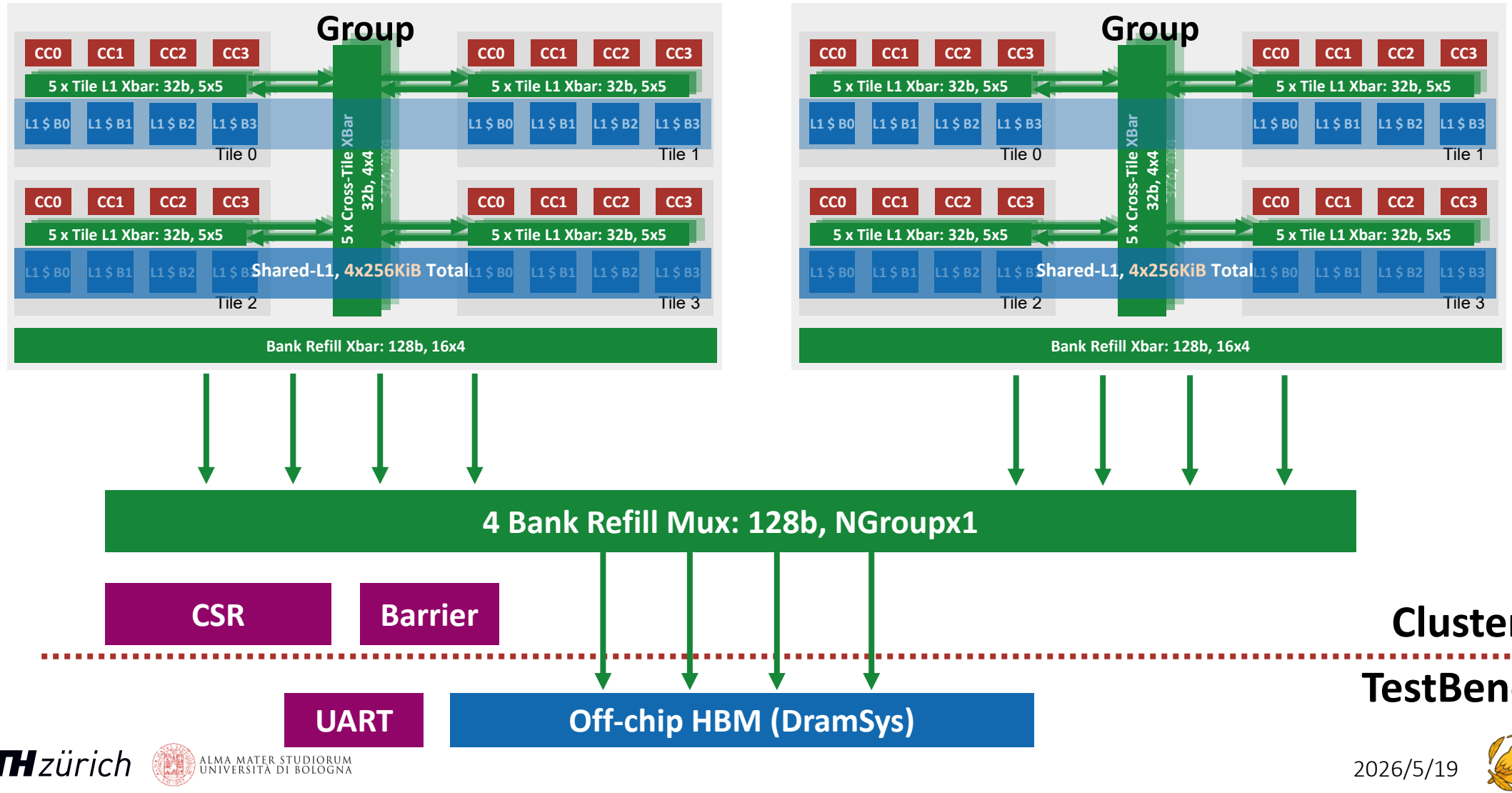
@pulp_platform

pulp-platform.org

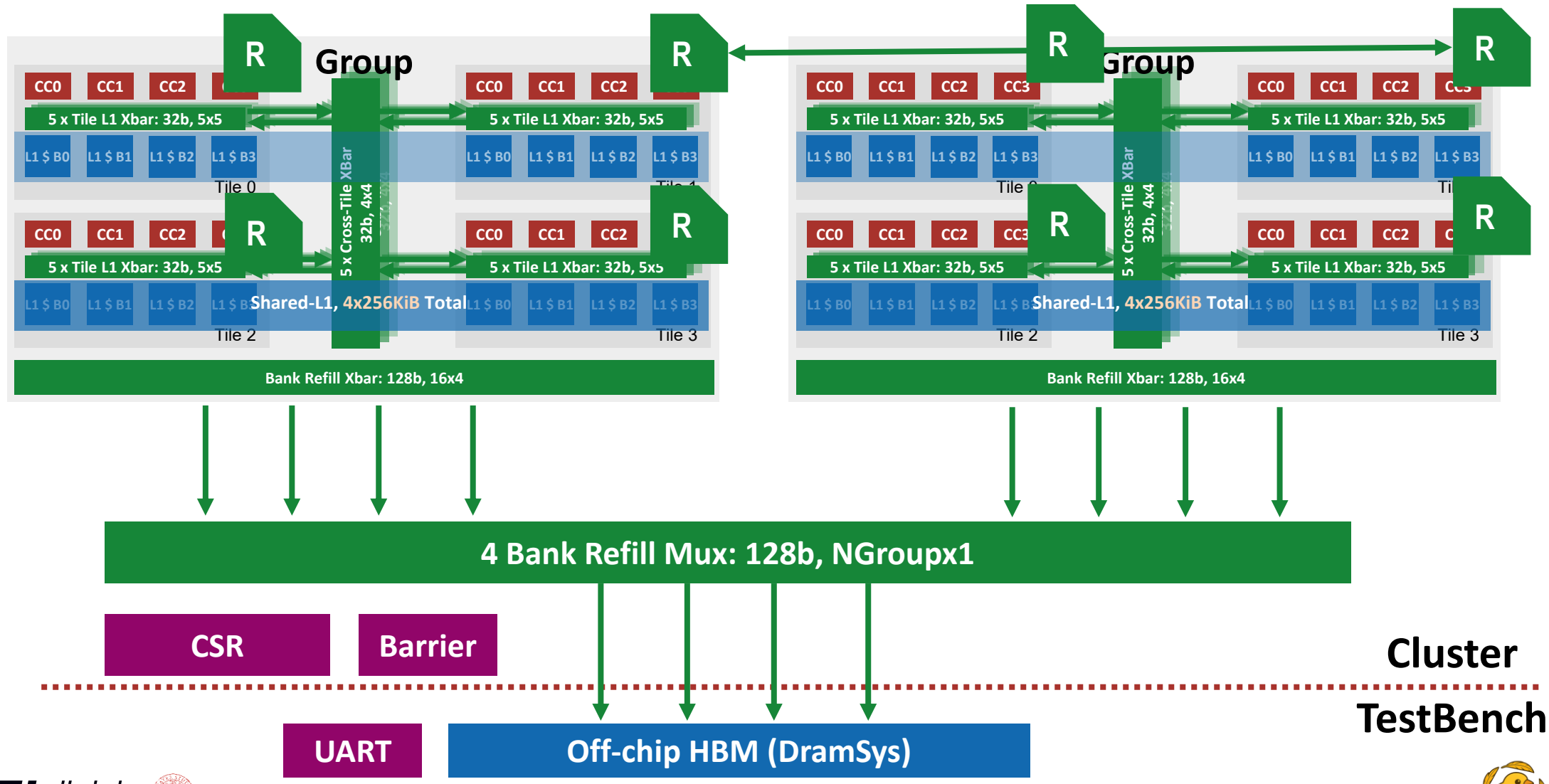
youtube.com/pulp_platform

PULP Platform
Open Source Hardware, the way it should be!

Hardware Development (Cluster)



Hardware Development (Cluster)



Hardware Development (Cluster)

- **Group NoC Wrapper**
  - Connect routers
    - Parameterize the number of router per tile
    - Currently configure to 1
  - Now testing and verifying 2x1 (2 groups) and 2x2 (4 groups) configurations
    - Most tests seem OK
    - Some small bugs in FFT
  - AXI ID exceeds limits for larger size
    - Mainly on the refilling xbar (will be replaced by NoC), and peripherals (not performance critical)
    - Working on to add id_remapper/id_serializer

Hardware Development (L1 Cache)

- Folded cache + forwarding buffer Rebasing
  - Rebase the dev branch to the main branch (xbar-based multi-tile stable version)
  - Fixed several config / env related issues, now can run some simple kernels, still some issues to be solved
  - The pull request is opened
  - Next: After fixing the remaining issue, the backend evaluation of the multi-tile config w/ folded cache can start

Hardware Development (L1 Cache)

- Cache controller debug fixing
  - **Writeback FSM**: Writeback and Flush FSM was broken with the previous modification, debugged it and now the cache write back and flush works
  - **Forwarding buffer handshake**: Handshake logic between the forwarding buffer and the SRAM access FSM
  - **Forwarding buffer internal**
    - **Allow simultaneous writeback and new write**: absorption fixed a bug where the forwarding buffer incorrectly assumed these two operations could not happen at the same time, which could cause one memory update to be lost.
    - **Prevent duplicate SRAM reads during an in-flight request**: fixed a race condition where the controller could mistakenly start a second read for the same cache line before the first read had finished.
    - **Remove false signals when no write request is active**: cleaned up write-mask handling so temporary invalid signals do not accidentally interfere with cache hit detection logic.

Software Development

- Initially finished the new RLC kernel debug
  - Change of the new kernel
    - **From peak-throughput test to fixed-rate RLC model**
      The old kernel pushed packets as fast as possible; the new kernel adds rate control so producer and consumer run at a configured input/output data rate.
    - **More realistic RLC software behavior**
      The latest version separates packet sending and UE status-report handling, and adds extra header accesses, ACK handling, and RLC statistics updates.
  - Potential performance impact
    - **Lower measured peak throughput, but more realistic workload behavior**
      Because the new kernel intentionally inserts delays to match the target data rate, the total runtime becomes longer than the old free-running benchmark.
    - **Higher memory and synchronization overhead per packet**
      New counter updates, header accesses, ACK processing, and list operations add extra memory traffic and may increase cache/lock pressure.

Software Development

- Initially performance of the new RLC kernel
  - Single tile, 4 core config, 256KB shared cache, 4 banks, 4 channel ddr4
  - 2 producer + 2 consumer, 100 RLC packages
  - Cycle 37550 → 114829 (+205%)
  - Potential performance issue:
    - some of the newly added header read is done by scalar memcpy, instead using vector load and store.
    - The current flow control parameter of the producer and consumer side need to be further tuned

Software Development

- **Some variables and structs are can be opt out by compiler, should we keep them explicitly?**
  Some structures, such as dlsch_ind and ue_status_rpt_content, are mainly used to model memory accesses. Since their loaded values are not really used later, the compiler may optimize these loads away. Should we mark them as volatile, or is there another preferred way to preserve this memory traffic?

- **Is TestDataStru intended to model real debug/statistics traffic?**
  TestDataStru dfx is filled inside rlc_send_pkt(), but it is not read after the function returns. I would like to confirm whether this structure is only used to model per-packet temporary data movement, or whether some later logic should consume it.

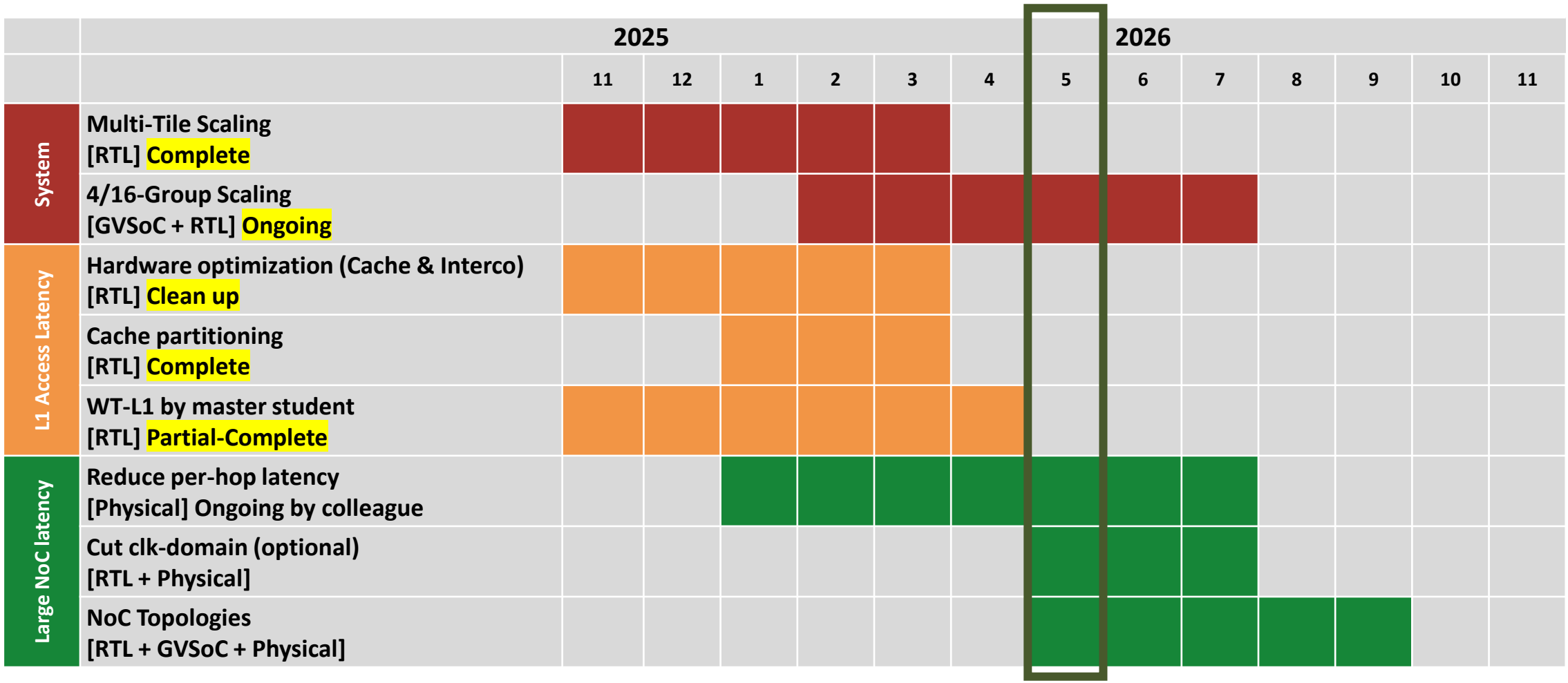
- **What is the intended meaning of firstSduPktRxCycle?**
  Currently, it is reset on every producer call, so it behaves more like a per-packet start timestamp rather than the first receive timestamp of the whole RLC flow. We should confirm whether this is intended.

- **Should some statistics become per-core or atomic in multi-consumer tests?**
  Fields such as pktdelay, pdcpcount, and rlcOm[] may be written by multiple consumers. This is fine for the current default test, but if we want to use more consumer cores, we may need to make these fields per-core, atomic, or protected by a lock.

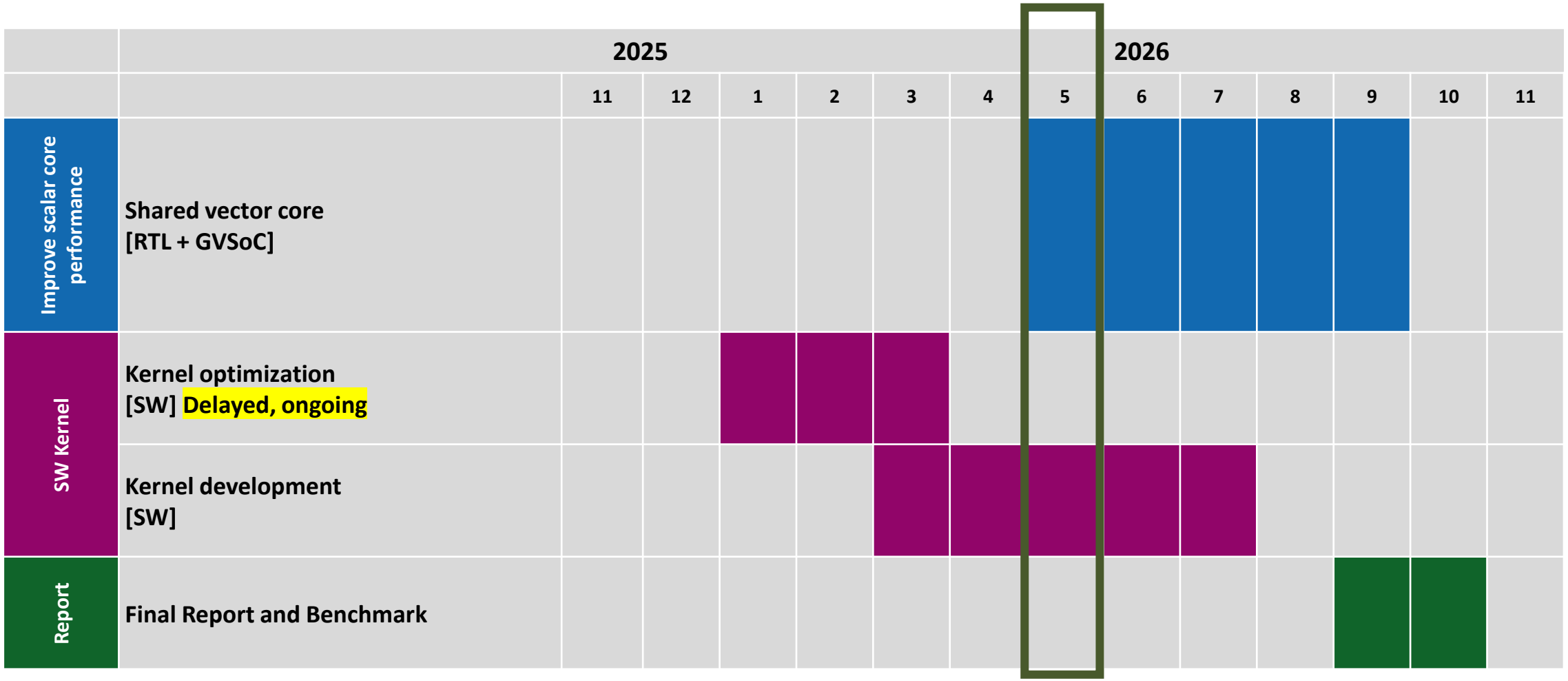
Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in May (ICCD). The progress on other side might be delayed
- **Diyou Side**
  - 7nm Flow (ongoing)
  - RTL development on multi-group
    Add router connection and router mesh
- **Zexin Side**
  - Continue rebase the folded cache+forwarding buffer branch to the multi-tile main branch
  - Continue the insitu cache GVSoC model
  - Fix the new RLC kernel bugs

Timeline


2025
2026
11
12
1
2
3
4
5
6
7
8
9
10
11
System
Multi-Tile Scaling
[RTL] Complete
4/16-Group Scaling
[GVSoC + RTL] Ongoing
L1 Access Latency
Hardware optimization (Cache & Interco)
[RTL] Clean up
Cache partitioning
[RTL] Complete
WT-L1 by master student
[RTL] Partial-Complete
Large NoC latency
Reduce per-hop latency
[Physical] Ongoing by colleague
Cut clk-domain (optional)
[RTL + Physical]
NoC Topologies
[RTL + GVSoC + Physical]

Timeline


2025
2026
11
12
1
2
3
4
5
6
7
8
9
10
11
Improve scalar core performance
Shared vector core
[RTL + GVSoC]
SW Kernel
Kernel optimization
[SW] Delayed, ongoing
Kernel development
[SW]
Report
Final Report and Benchmark

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Complete with some problems, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Performance degrade on single tile
    => Needs more work to support larger system

L1 Access Latency

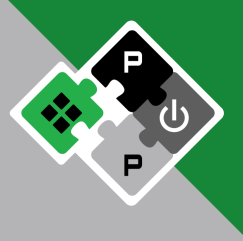
- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.
    Discussed Solution: critical section

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user



Thank you!